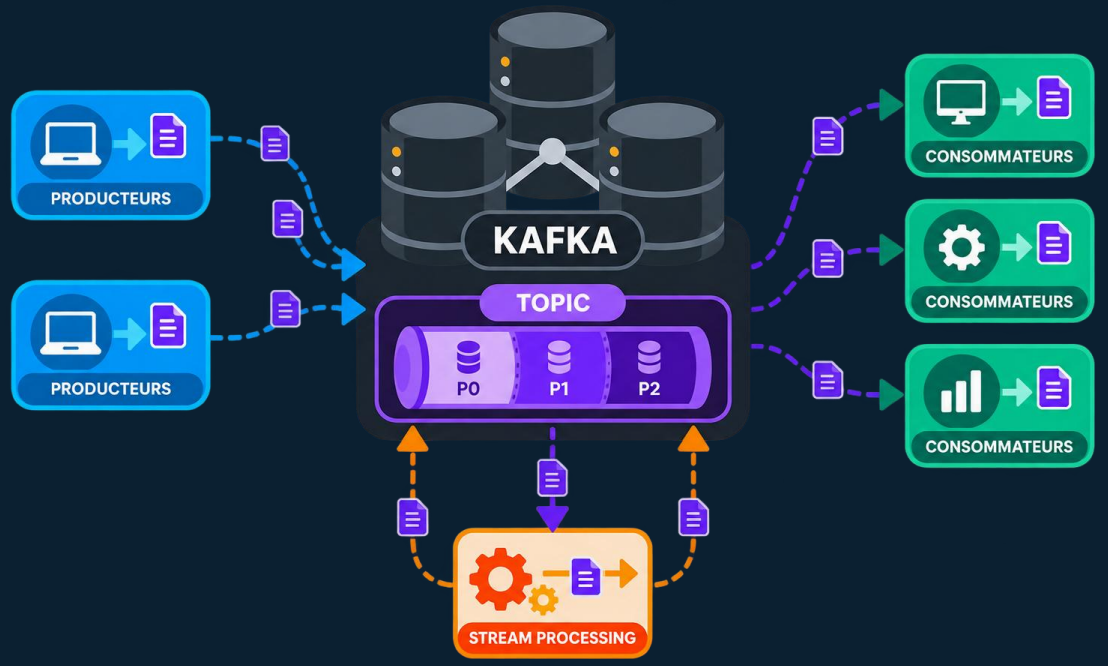
Traitement des flux et résilience EDA

Guide d'ingénierie Kafka

Du premier message publié à la résilience EDA



Un Sandbox d’apprentissage …
Spring Boot · Nuxt · Kafka · Flink · Hazelcast

Les meilleures pratiques de l'architecture EDA

Ce guide présente un projet concret (avec implémentation de style Sandbox) qui intègre les meilleures pratiques.

01 Sous-évaluation

Les équipes de réalisation sous-évalue généralement la complexité d'intégration avec EDA.

02 Kafka distribuée

La nature distribuée de l'architecture Kafka doit être prise en considération.

03 Partenaires externes

Nos partenaires ont très peu d'expertise dans ce domaine – C'est simplement un constat.

04 Projet style Sandbox

Un projet concret qui démontre une implémentation d'architecture EDA.

05 Introduction et présentation des concepts avec implémentation

Nous allons présenter les meilleures pratiques pour mettre en place une architecture résiliente avec des exemples concrets – Via le Sandbox d'apprentissage.

Plus de 40 diapositives de parcours + quelques annexes de repérage et d'approfondissement.

Six étapes pour le transfert des connaissances

Un même fil conducteur : publier une requête, produire un effet métier, puis contrôler/corroborer cet effet.

01 Découvrir le Sandbox

Application, données et parcours d'une requête (message).

02 Comprendre EDA/Kafka

Topics, partitions, clés et ordre métier.

03 Fiabiliser les effets

Livraison, transactions, idempotence, DLT et Outbox.

04 Explorer Flink

Checkpoint, savepoint et frontières de runtime.

05 Exploiter et distribuer

Parallélisme, arrêt, sondes et élasticité.

06 Prouver et choisir

Observabilité, gouvernance, corroboration et TCO/ROI.

Projet GITHUB : https://github.com/rdemers/org-rd-fullstack-springboot-eda

Publier un message ne suffit pas à construire une plateforme EDA

EDA : architecture événementielle. Le transport rend l'échange possible; il ne garantit pas, seul, la fiabilité des effets.

01 **Transport**

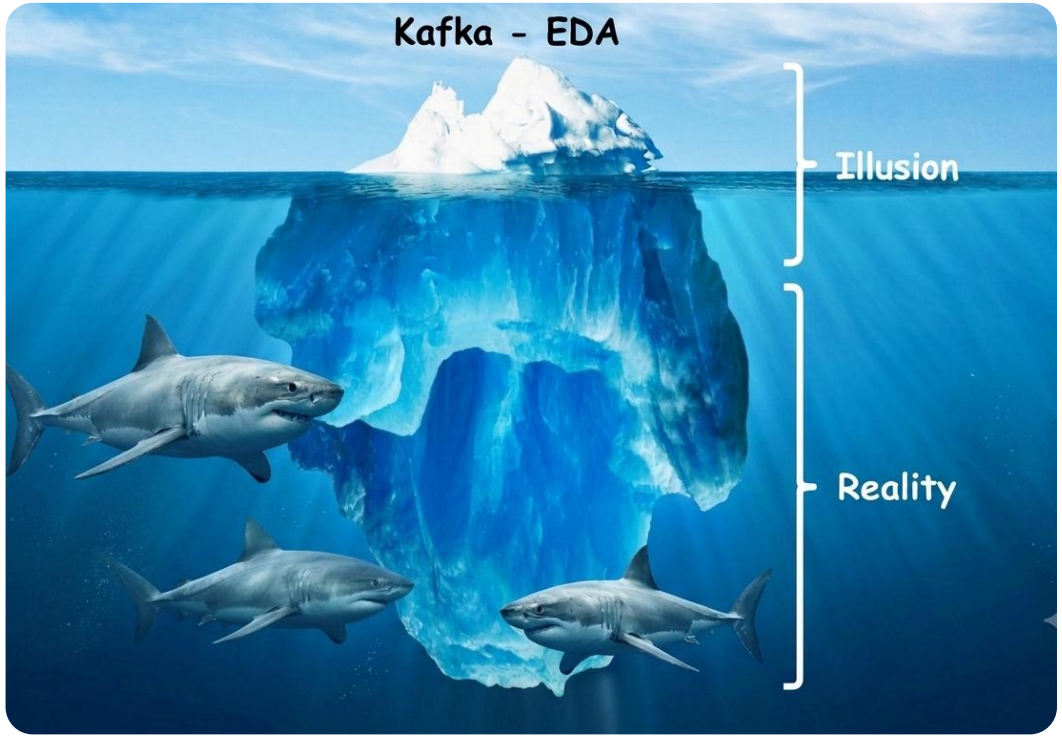
Publier, conserver et consommer les messages normalisés.

02 **Exactitude**

Ordonner, dédupliquer et valider les effets (idempotence).

03 **Confiance**

Observer, gouverner, contrôler et corroborer les données.



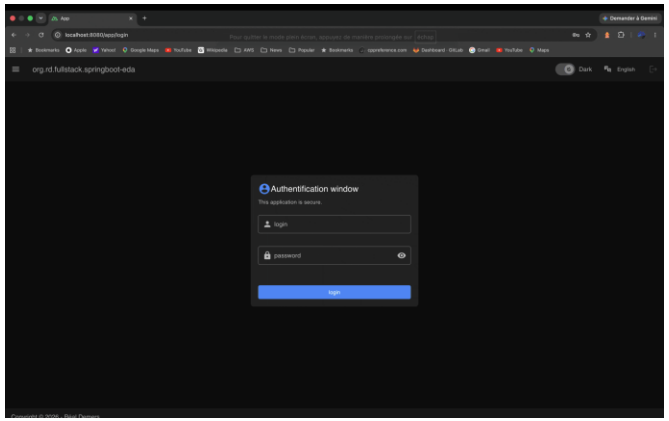
Nos requins : Bronze, argent et or – Nos spécialistes de l’architecture médaillon.

Une application autonome pour observer les effets/compromis/décisions

Le frontend statique Nuxt et le backend Spring Boot sont regroupés dans un seul artefact déployable.

Navigateur

Interface Nuxt



ARTEFACT APPLICATIF AUTONOME

Spring Boot + Nuxt

API BFF et ressources statiques

Configuration, transactions
et logique métier

Kafka

Flink

Hazelcast

HSQLDB

Sandbox éducatif : une commodité d'exécution, pas une topologie de production.

Une requête relie une personne, un produit et un inventaire

Le traitement doit faire évoluer le solde, l'inventaire et le résultat de la requête de façon cohérente.

01 Clé, Latence, verrou

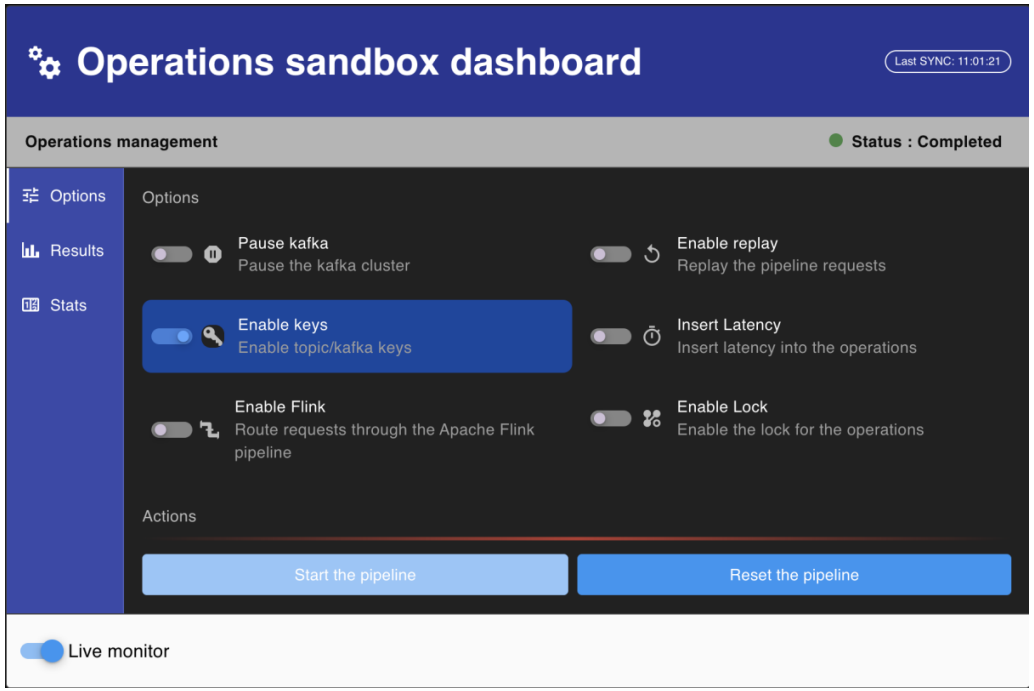
Rendre visible l'effet du parallélisme.

02 Pause, Rejeu

Voir la progression et la reprise.

03 Option Flink

Alternative Flink pour le traitement.



PERSON

L'acheteur et son solde

PRODUCT

Le produit et son prix unitaire

REQUEST

Opérations à effectuer

INVENTORY

L'inventaire disponible

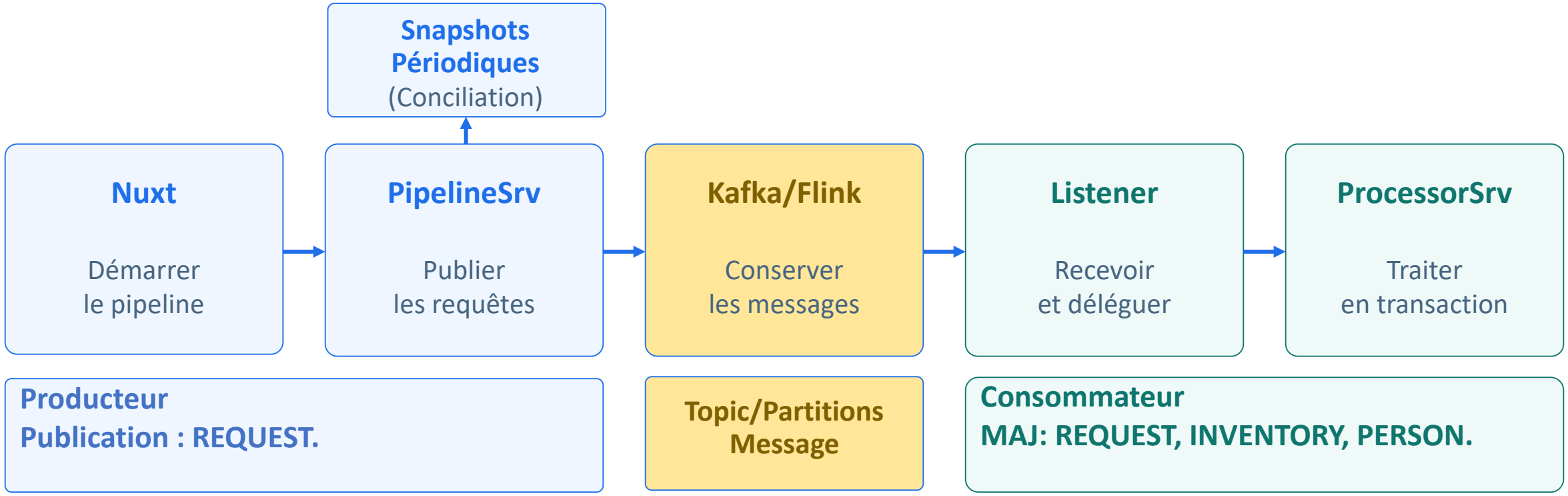
JRN_EVENT

Consommation
Des évènements (idempotence)

Invariant du parcours : l'inventaire ne doit jamais devenir négatif. L'écran des opérations est le poste d'expérimentation.

Suivre une requête(message) de bout en bout

Chaque composante porte une responsabilité distincte; les frontières de validation viendront ensuite.



Une requête reçue n'est pas immédiatement un effet métier validé.

Normalisation des données

Le topic transporte les messages d'un producteur. Un message possède une charge utile et des métadonnées. La normalisation ne doit pas dépendre de la technologie et doit être compréhensible par l'ensemble des consommateurs présents et futurs.

01 Normalisation

La normalisation vise à rendre accessible/compréhensible le message aux consommateurs – Durable dans le temps.

02 Métadonnées

Permet au consommateur de connaître le contexte, d'obtenir l'identifiant unique. Alternative, Hash du payload … S'il y a un champ temporel.

03 Intégrité

Un consommateur doit pouvoir valider l'intégrité du message AVANT le traitement.

Très important. Non-falsifiable.
Principe du « Zéro Trust ».

Devrait être des UUID.
OK pour un Sandbox.

```
{
    "header":{
        "eventId":"IFSD-5ZHM-HO4A-D6AA-AAAA-AAAA-24C2",
        "version":"1.0",
        "batchId":"0e9b2243-b7fc-4926-8a77-e50fcaeb1562",
        "replayId":"<none>"
        },
    "payload":{
        "requestId":0,"personId":3,"productId":5,
        "qty":10,"operation":10,"result":10
        },
    "footer":{
        "signature":"55c2be8d5a4 ... 01d9ddf192fac2e9b4"
        }
}
```

Technologie: les métadonnées ne doivent pas dépendre des fonctionnalités spécifiques d'un MOM.

Ni topic universel, ni multiplication sans intention

Il faut chercher un équilibre autour des agrégats, des responsabilités et des contrats.

Fat pipe



Un topic pour des événements hétérogènes.

Risque : contrat flou et clé peu naturelle (ou impossible).

Complique exponentiellement la consommation des messages.

Thin pipes



Des topics spécialisés par type.

Compromis : ordre inter-topics et exploitation.

Idéal pour la sélection d’une clé de partition.

Par agrégat

Une intention et un contrat explicites.

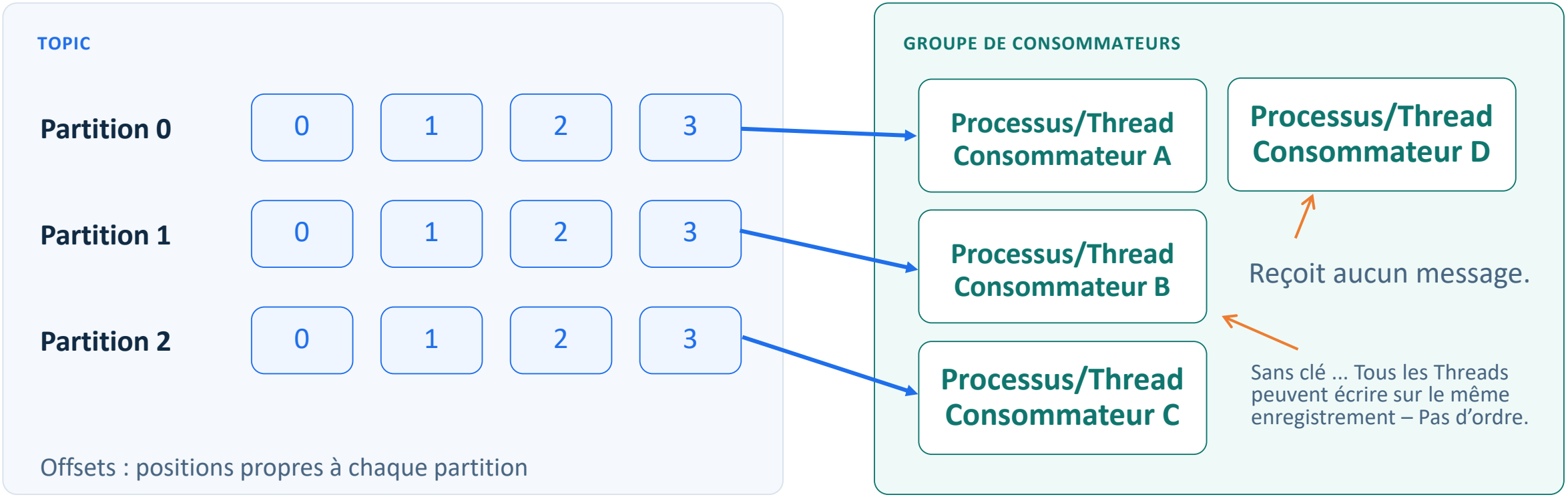
Regrouper les événements dont l'ordre est lié.

Peut-être difficile de déterminer une bonne clé de partition.

Un topic est un contrat intentionnel, pas seulement un tuyau de transport.

Un topic Kafka … Les partitions allouent l’élasticité horizontale

Le topic organise les messages; la partition donne l'ordre (si clé); l'offset repère une position; le groupe partage le travail.

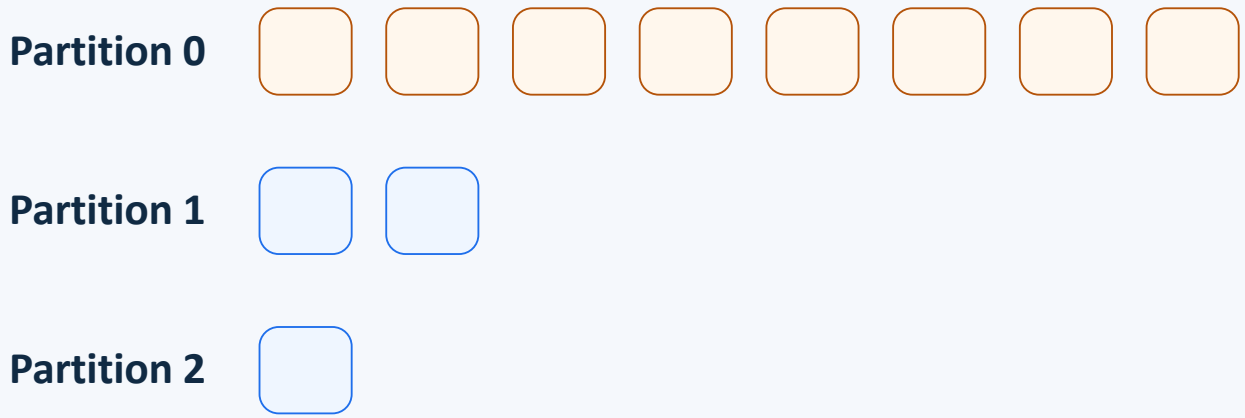


L'ordre est garanti que dans une partition (si clé de partition), pas globalement entre toutes les partitions.

Une clé déséquilibrée peut neutraliser l’élasticité horizontale

Le débit peut rester limité par une seule partition, même lorsque les autres consommateurs sont disponibles.

SCHÉMA DE DÉSÉQUILIBRE



Une clé dominante concentre le travail.

01 **Mesurer la distribution**

Observer le lag par partition, pas seulement son total.

02 **Localiser le goulot**

Clé, verrou, base ou ressource externe.

03 **EKS – Rebalance Kafka**

Un pod inactif (sous actif) est retiré, causant un rebalance des partitions vers s’autres pods en continu ... Aïe !

Répartir davantage n'est utile que si l'ordre métier et les invariants restent préservés.

La clé Kafka doit exprimer l'ordre métier recherché

Dans le projet, la clé est « productId » permet un ordre des messages pour l'invariant de l'inventaire.

Sans clé

Un même produit peut apparaître dans plusieurs partitions.

Plusieurs threads peuvent agir sur le même inventaire.

Avec « productId » comme clé

Les requêtes du produit rejoignent une même partition.

Le chemin direct conserve un traitement séquentiel par produit.

Cependant, cet ordre ne permet pas d’adresser la synchronisation du solde de la personne.

Décision de conception : ordonner les événements là où le métier l'exige.

Une seule option change; l'invariant peut basculer

Les deux rapports de la documentation du projet utilisent le même inventaire initial et les mêmes requêtes, avec ou sans clé.

AVEC CLÉ

Banana : 0

ID	Code	Description	Qty/Qté
0	Apple	Mcinstosh Apple	0
1	Banana	Banana split	0
2	Steak	T-Bone Steak 4pack	10
3	Oats	Quaker Quick Oats	80

Résultat illustré : inventaire non négatif.

SANS CLÉ ou clé sur « personId ».

Banana : -10



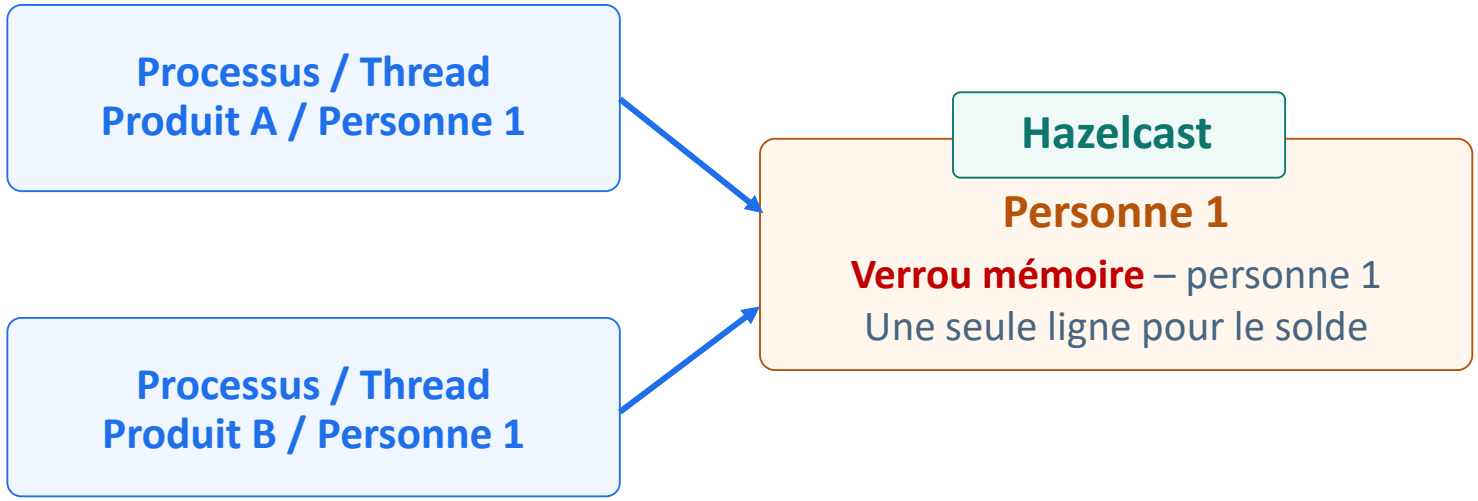
ID	Code	Description	Qty/Qté
0	Apple	Mcinstosh Apple	120
1	Banana	Banana split	-10
2	Steak	T-Bone Steak 4pack	160
3	Oats	Quaker Quick Oats	170

Résultat illustré : survente silencieuse.

Extraits documentaires : la comparaison doit porter sur les effets métier, pas seulement sur le flux des données.

Protéger la personne et le produit

Le verrou Hazelcast est indexé sur PERSON_ID; la ligne INVENTORY est ordonnée par PRODUCT_ID.



La course

Lire le solde la personne
Vérifier le solde et le modifier.

Deux traitements peuvent passer la même vérification du solde.

Ajouter « personId » à la clé actuelle ne va pas fonctionner !

Le périmètre d'une protection doit correspondre à l'invariant qu'elle cherche à protéger.

Alternative - Protéger PERSON ou INVENTORY via verrouillage SQL

L’approche SQL ... Alternative qui ne permet pas de conserver l’ordonnancement.

Verrouillage pessimiste

Verrouiller la ligne à la lecture pour que les autres attendent. En SQL, SELECT ... FOR UPDATE.

Processus en attente de la base de données. « sleep mode ».

Plus d’élasticité et impact sur le cycle de vie d’un pod.

Verrouillage optimiste



Une colonne @Version ; la mise à jour ne réussit que si la version est inchangée, sinon l'application doit refaire une tentative.

Tourne en rond ... Consomme les ressources $$$. Lutte entre les pods (Processus/Thread) !!!

Mise à jour conditionnelle

Déléguer la vérification du solde dans l'écriture de la base de données (where clause).

La logique métier descend dans la requête SQL – Brise la cache JPA.

Protège seulement les invariants explicitement de la garde sans protéger automatiquement l'ensemble de l'entité contre toutes les modifications concurrentes.

Brise l’élasticité horizontale, génère du DLT, consomme des ressources et peut causer des reprises à répétition.
Cette alternative ne permet pas de conserver l’ordonnancement !!!

Trois garanties, mais toujours une frontière à préciser

La question n'est pas seulement combien de fois un message arrive, mais quels effets deviennent visibles.

Garantie	Principe	Compromis
At-most-once	Valider la position avant le traitement.	Une perte est possible et va survenir !!!
At-least-once 	Valider après le travail réussi.	Des doublons sont possibles. Il faut mettre en place l'idempotence.
Exactly-once 	Résultats Kafka et offsets sources atomiques.	**Les effets externes ne sont pas inclus.** Att: checkpoint !

```java
public class KafkaPipelineListener {

    @KafkaListener(id = KafkaConstants.CST_LISTENER_PROCESSOR,
                   topics = KafkaConstants.CST_TOPIC_KAFKA_REQ,
                   groupId = KafkaConstants.CST_TOPIC_GROUP,
                   containerFactory = "kafkaListenerContainerFactory")
    public void listen(ConsumerRecord<String, String> record, Acknowledgment
                @Header(KafkaHeaders.RECEIVED_TOPIC)     String topic,
                @Header(KafkaHeaders.RECEIVED_PARTITION) int partition,
                @Header(KafkaHeaders.OFFSET)             long offset) throws

        // Per-record processing (per-client lock + transactional processor
        // is shared with the Flink sink path; see PipelineSrv.handle(). An
        // the container's DefaultErrorHandler, which retries with a bounde
        // routes the record to the DLT. The JPA transaction in the process
        // so no partial DB state is committed.
        pipeline.handle(record.value());

        // Commit the offset only after a successful, committed processing.
        // (at-least-once: a crash before this point replays the message.)
        ack.acknowledge();
    }
```

At-least-one.

Choix du projet Sandbox : at-least-once + idempotence du traitement métier.

Trois mécanismes différents travaillent ensemble

Idempotence du producteur, transaction Kafka et isolation du consommateur ne répondent pas à la même question.

Producteur idempotent

Éviter les doublons dus aux nouvelles tentatives du producteur.

enable.idempotence=true

Transaction Kafka

Valider ou annuler un ensemble de publications dans Kafka.

commit / abort

Consommateur isolé

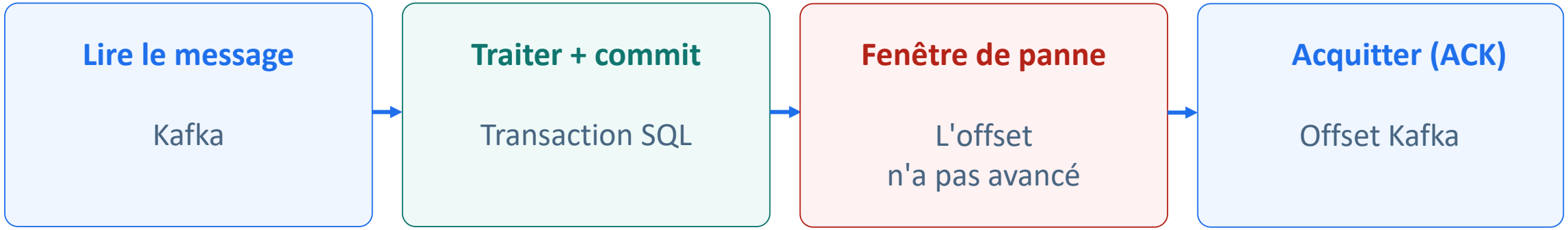
Messages transactionnels : lire seulement après validation.

isolation.level= read_committed

Ces mécanismes Kafka ne remplacent pas l'atomicité ni l'idempotence de l'effet SQL.

Le commit métier doit précéder l'acquittement

Une panne entre les deux validations provoque une relivraison : c'est précisément le rôle de l'idempotence.



Offset avant SQL

Risque : l'effet métier peut être perdu.

- Panne/Arrêt – Pas de relivraison.
- Exception – Pas de relivraison.

SQL avant offset

Risque : l'effet peut être tenté de nouveau.

- Panne/Arrêt – Relivraison.
- Exception – Relivraison.

Un doublon de livraison doit devenir un non-événement pour les données – Le traitement DOIT ÊTRE IDEMPOTENT.

Reconnaître l'événement (le message) – Idempotence

Un même événement livré plusieurs fois ne doit appliquer l'effet métier qu'une seule fois.

UNE TRANSACTION SQL

1 **Vérifier / enregistrer l'identité**

JRN_EVENT : (CONSUMER_ID, EVENT_ID)

2 **Appliquer puis enregistrer l'issue**

INVENTORY + solde + REQUEST + JRN_EVENT

Une contrainte d'unicité reste l'autorité de déduplication face à la concurrence. Il ne constitue pas le commit. Garder distincts l'identité, le type de traitement et son résultat.

```java
public class ProcessorSrv {
    public ProcessorSrv(JrnEventRepository jrnEventRepository, InventoryRepository inventoryRepository,

    @Transactional(propagation = Propagation.REQUIRED, isolation = Isolation.READ_COMMITTED)
    public void process(EventMessage<EventHeader, Request, EventFooter> message,
        boolean latence) throws Exception {

        SanityResult sanity = sanityCheck(message);
        if (! sanity.proceed())
            return; // Already processed, or exception thrown (rollback and retry).

        final Request  request  = message.getPayload();
        final JrnEvent jrnEvent = sanity.jrnEvent();

        // Get inventory for the product.
        Optional<Inventory> inventoryOptional = inventoryRepository.findByProductId(request.getProductId());
        if (inventoryOptional.isEmpty()) { // Big problem here.
            finalizeOutcome(request, jrnEvent, Result.ERROR, false);
            return;
        }
```

Une seule transaction.

Vérifier l'idempotence.

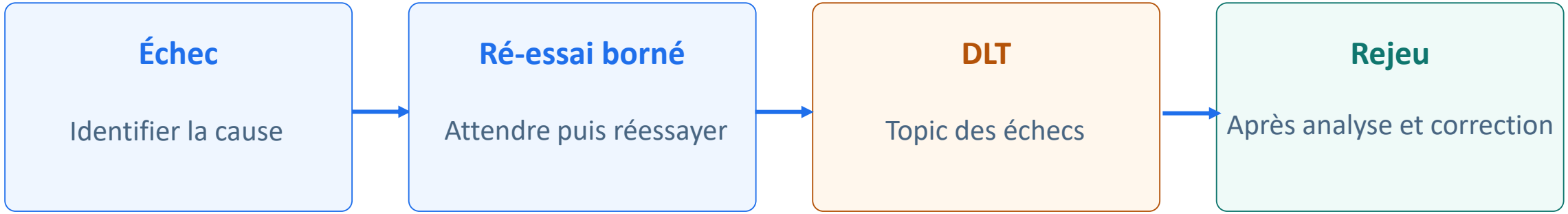
Référence de la journalisation pour la mise à jour.

Afin ... Début de la logique d'affaires.

Marqueur de déduplication et effets métier : valider ensemble, ou annuler ensemble.

Une erreur doit suivre un parcours explicite

Réessayer un incident transitoire, isoler un échec persistant, puis organiser un rejeu contrôlé.



Contexte

Identité de l'événement, topic, partition, offset et cause de l'échec.

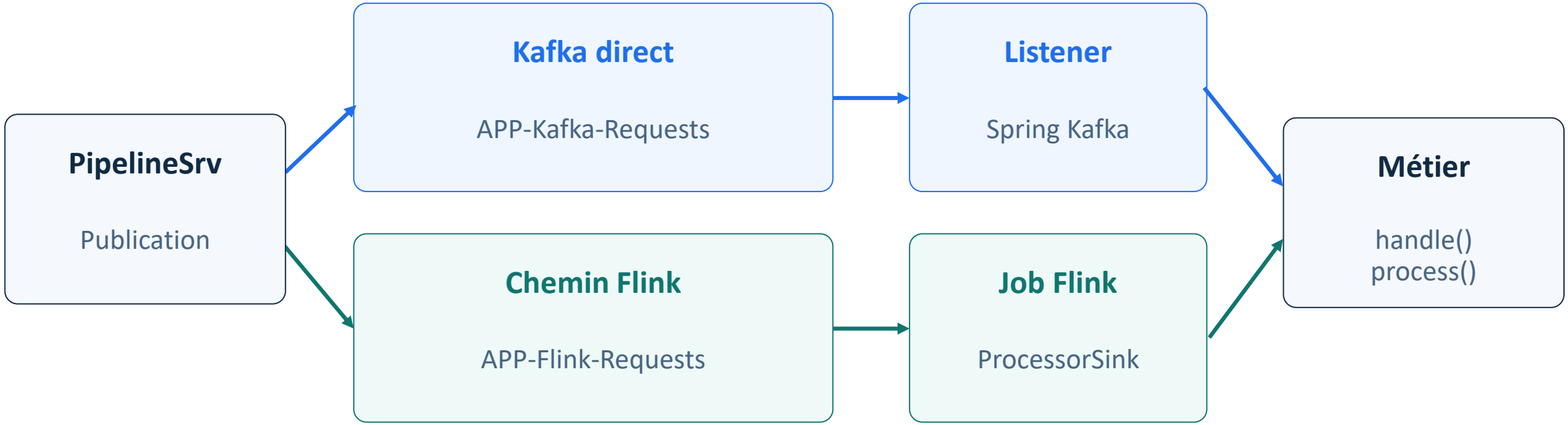
Procédure

Comment faire le rejeu … Un message en DLT n’est pas un état final.

Une DLT doit avoir un responsable et une procédure de traitement (Ne pas transformer le DLT en oubliette).

Flink change le transport, pas l'intention métier

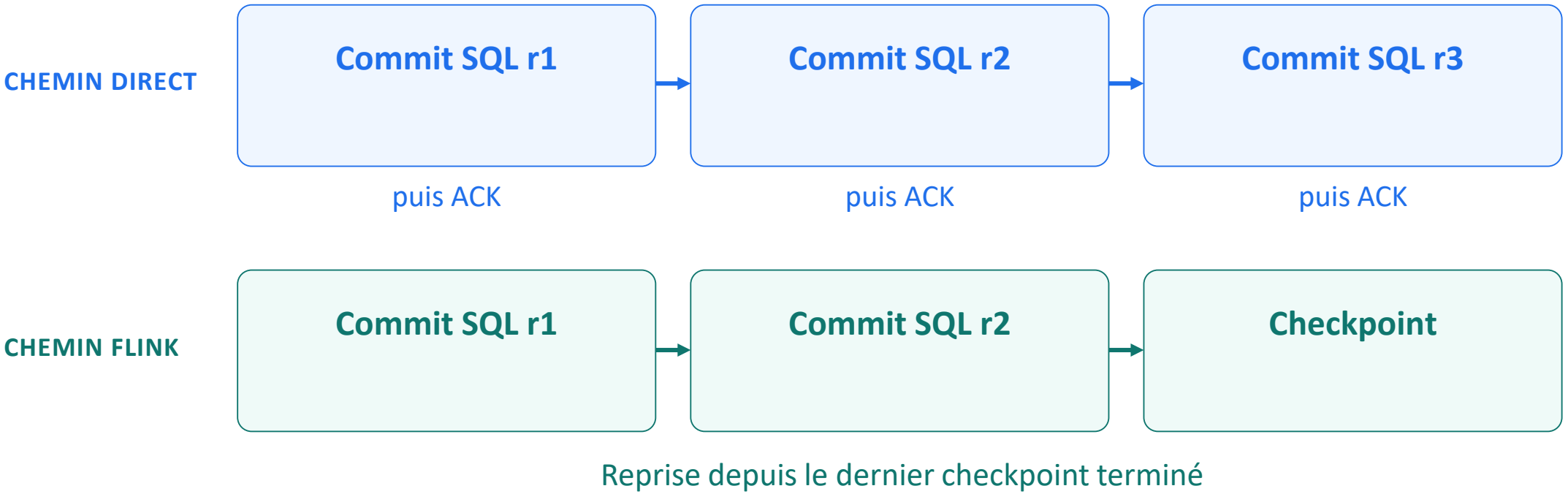
Le bouton Flink permet de choisit le topic de publication; les deux chemins convergent vers le même traitement.



Un même effet SQL, mais des frontières de progression et de reprise différentes.

Un checkpoint Flink n'est pas un commit SQL

Le dernier checkpoint terminé fixe le point de reprise Flink, pas l'état déjà validé dans SQL.



Note : l'idempotence doit aussi être considérée pour les compteurs et les effets secondaires (Voir Dashboard).

Le pont in-JVM n'est pas une intégration distribuée

Un TaskManager distant ne partage ni les beans Spring, ni les repositories JPA, ni l'état statique de l'application.

Sandbox local

MiniCluster et Spring partagent une JVM.

Le pont résout le service à l'exécution.

Le chemin value-only abandonne clés et en-têtes.

Architecture distribuée

Séparer les runtimes.

Partager des règles Java, pas des beans actifs.

Choisir un connecteur ou un contrat distant.

Choisir le modèle de déploiement avant de choisir le mode d'intégration.

Le nombre de partitions fixe les limites de l’élasticité horizontale

Ajouter des pods n'ajoute du parallélisme que s'il reste des partitions à leur attribuer.

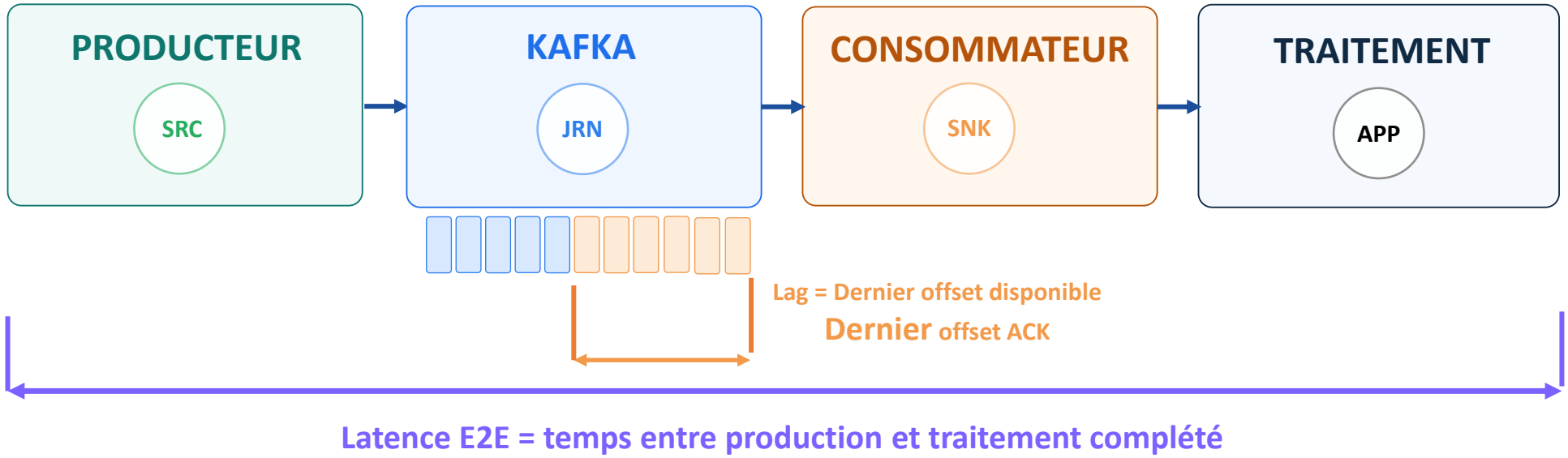
Parallélisme utile = min(partitions, pods × threads par pod)

Exemple : 8 partitions	Threads demandés	Threads actifs au maximum
1 pod × 2 threads	2	2
4 pods × 2 threads	8	8
5 pods × 2 threads	10	8 (2 threads inactifs)

Les threads excédentaires ne créent ni partitions ni capacité utile.

Latence et Lag … La bête noire de l'EDA

Deux mesures complémentaires : le lag indique un retard en volume; la latence indique l'impact temporel réel.



À retenir : Lag = Retard en volume messages/offsets. Latence = Retard dans le temps de traitement. Corrélés, mais non équivalents.

Changer les consommateurs change la propriété des partitions

Élasticité, redémarrage et défaillance font partie du fonctionnement normal d'un groupe.

Déclencheurs

Un pod arrive.
Un pod s'arrête.
Un consommateur dépasse un timeout.

Effets

Réaffectation des partitions.

Interruption possible et reprise depuis la position validée.

Protections

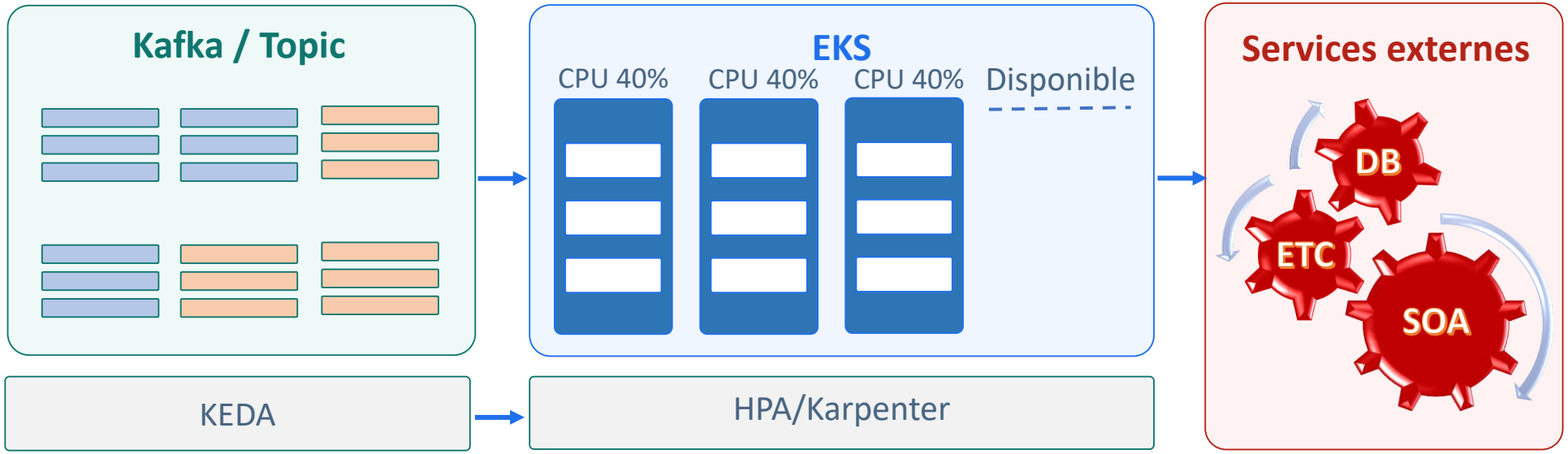
Traitement idempotent.
Lots et timeouts adaptés.
Élasticité stabilisée.
Stratégie de groupe adaptée.

Réaffectation des partitions
La réaffectation va acheminer des reprises (doublons).

Un système élastique doit savoir changer de propriétaire sans corrompre l'état.

L’élasticité horizontale EKS/EDA

L’élasticité horizontale des consommateurs EDA repose sur la connaissance du LAG.



Élasticité horizontale

HPA CPU/mémoire échoue : lag élevé sans CPU élevé, plafond de partitions ignoré, pas de d'élasticité-à-zéro.

HPA/Karpenter gère les instances machine, KEDA le lag et la consommation.

Mettre en pause n'est pas arrêter le consommateur

Pour suspendre temporairement le travail sans quitter le groupe, le sandbox utilise pause() / resume().

Une console pour contrôler les opérations n’est pas une option !!!

Action	Consommation	Appartenance au groupe
pause()	Suspendre la consommation.	Le consommateur reste membre.
resume()	Reprendre la consommation.	La même instance continue.
stop()	Arrêter le conteneur.	La sortie déclenche un rebalance (reprises possibles).
start()	Démarrer le conteneur.	L’entré déclenche un rebalance (reprises possibles).

Kafka : pause légère du conteneur. Flink/Sandbox : arrêt avec savepoint (checkpoint). Un rebalance déclenche des reprises !!!

L'arrêt fait partie des ajustements du système

Le temps de grâce doit permettre de terminer le travail en cours et de fermer les ressources dans le bon ordre.

1
Retirer le trafic
Readiness / drainage

→

2
Signaler l'arrêt
preStop / SIGTERM

→

3
Terminer l'en-cours
Commit / Ack

→

4
Fermer les ressources
Avant la fin de grâce

```
#
server:
  shutdown: graceful
#
  lifecycle:
    timeout-per-shutdown-phase: 45s
#
```

application.yml

Arrêt en cours.

```java
public class KafkaPipelineListener {
    @KafkaListener(id = KafkaConstants.CST_LISTENER_PROCESSOR,
                   topics = KafkaConstants.CST_TOPIC_KAFKA_REQ,
                   groupId = KafkaConstants.CST_TOPIC_GROUP,
                   containerFactory = "kafkaListenerContainerFactory")
    public void listen(ConsumerRecord<String, String> record, Acknowledgment ack,
                       @Header(KafkaHeaders.RECEIVED_TOPIC)     String topic,
                       @Header(KafkaHeaders.RECEIVED_PARTITION) int partition,
                       @Header(KafkaHeaders.OFFSET)             long offset) throws Exception {

        // If the application is shutting down, stop processing immediately. The record is
        // neither processed nor acknowledged, so its offset is not committed and it will be
        // redelivered after restart (at-least-once).
        if (!smartLifecycleSrv.isRunning()) {
            logger.warn("Application is shutting down – skipping record - topic={}, partition={}, offse
                topic, partition, offset);
            return;
        }

        // Per-record processing (per-client lock + transactional processor + completion count)
        // is shared with the Flink sink path; see PipelineSrv.handle(). Any exception propagates to
        // the container's DefaultErrorHandler, which retries with a bounded back-off and ultimately
        // routes the record to the DLT. The JPA transaction in the processor rolls back on failure,
        // so no partial DB state is committed.
        pipeline.handle(record.value());

        // Commit the offset only after a successful, committed processing.
        // (at-least-once: a crash before this point replays the message.)
        ack.acknowledge();
    }
```

Pour les producteurs et consommateurs Kafka, l'arrêt gracieux est une exigence critique, pas une optimisation optionnelle.

L'arrêt propre réduit les reprises; l'idempotence rend les reprises possibles.

Trois sondes répondent à trois questions différentes

Une panne d'accès au service et un processus irrécupérable ne demandent pas la même réaction.

Startup

Le démarrage terminé ?

Ne pas juger trop tôt une JVM en initialisation.

Liveness

Le processus est-il sain ?

Un échec peut provoquer son redémarrage.

Readiness

Peut-il recevoir du trafic ?

Un échec retire le pod des endpoints du service.

```yaml
management:
  server:
    port: 8081
  endpoints:
    web:
      exposure:
        include: "info,health,prometheus"
  prometheus:
    metrics:
      export:
        enabled: true
```

Fichier : application.yml.

Pour les producteurs et consommateurs Kafka.
La gestion de l’état interne est une exigence critique.

Santé : une décision rapide. Métriques : une mesure de comportement dans le temps.

Un contrat transforme un topic en une interface compréhensible

La gouvernance rend explicites le sens des données, les responsabilités et les conditions de leur usage.

Sens métier

Définition et exemples d'événements.

Propriété

Équipe responsable et soutien.

Schéma

Version et règles de compatibilité.

Dépendances

Producteurs, consommateurs et lignage.

Service

Fraîcheur, disponibilité et qualité.

Protection

Accès, classification et rétention.

Un topic sans contrat ni responsable crée une dépendance difficile à exploiter.

La propriété et la plateforme sont complémentaires

Le guide ne présente pas Data Mesh et Data Fabric comme deux produits concurrents.

Data Mesh : qui et pourquoi ?

Propriété par domaine.
Données comme produit.
Plateforme libre-service.
Gouvernance fédérée.

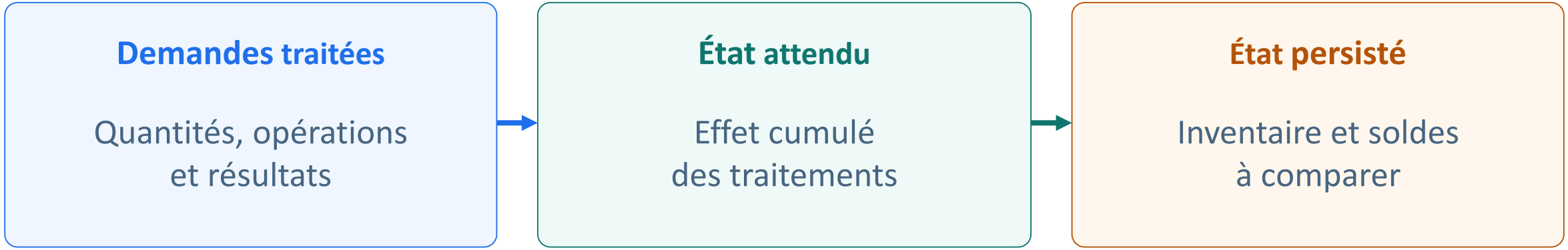
Data Fabric : comment relier ?

Intégration et connecteurs.
Catalogue et lignage.
Politiques et qualité.
Accès et observabilité partagés.

Kafka peut soutenir ces approches; il ne remplace ni la propriété ni la gouvernance.

Un message consommé ne prouve pas que l'état est juste

La corroboration compare des représentations des mêmes faits et recherche les divergences.



Contrôle concret : les requêtes exécutées se réconcilient-elles avec les mouvements de l’inventaires et le solde du client ?

Idempotence : ne pas appliquer deux fois. Corroboration : vérifier le résultat global (l’intégrité).

Conciliation des données en EDA (1/2)

La livraison d'un événement ne garantit jamais que les systèmes restent cohérents entre eux.

01 Fréquent

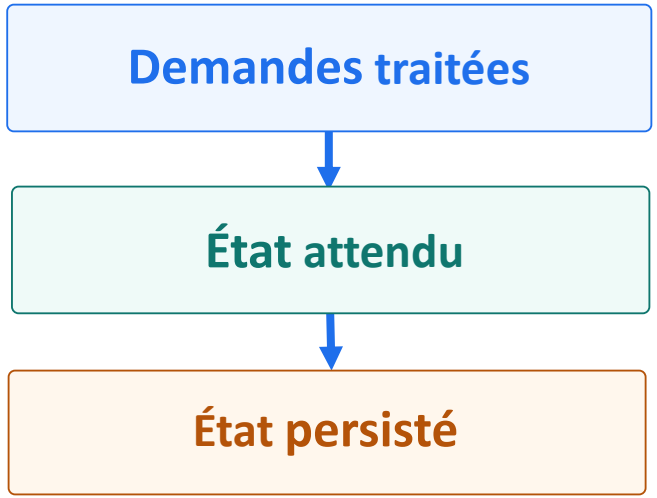
Comptages des résultats.
Sommes de contrôle.

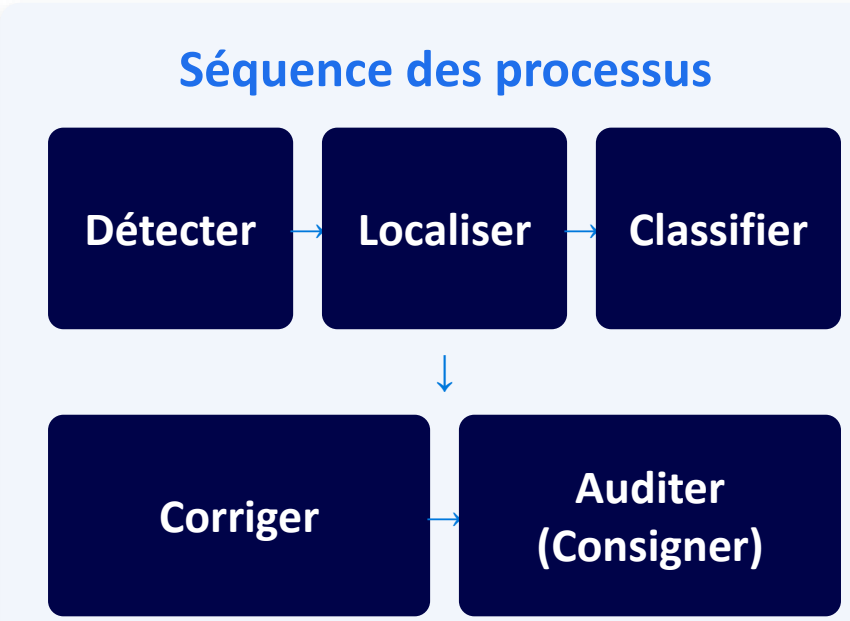
02 Périodique

Snapshots d'état.
Invariants métier.
Comparaison attendu/réel.

03 Approfondi

Rejeu de réconciliation.
Consommateur fantôme.
Audit des écarts.



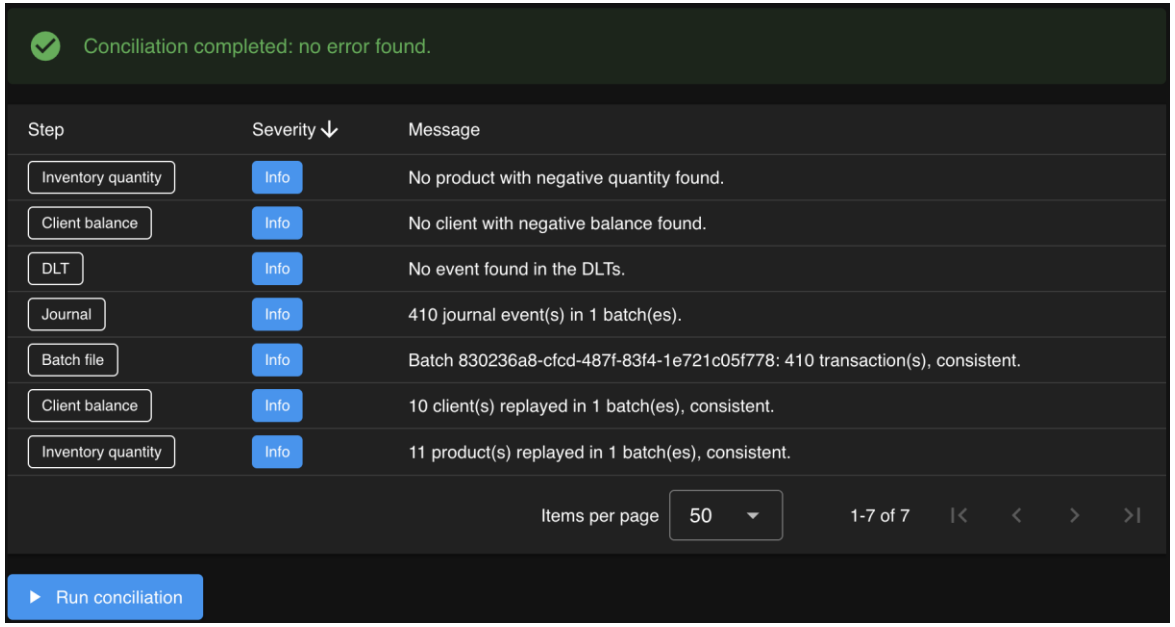


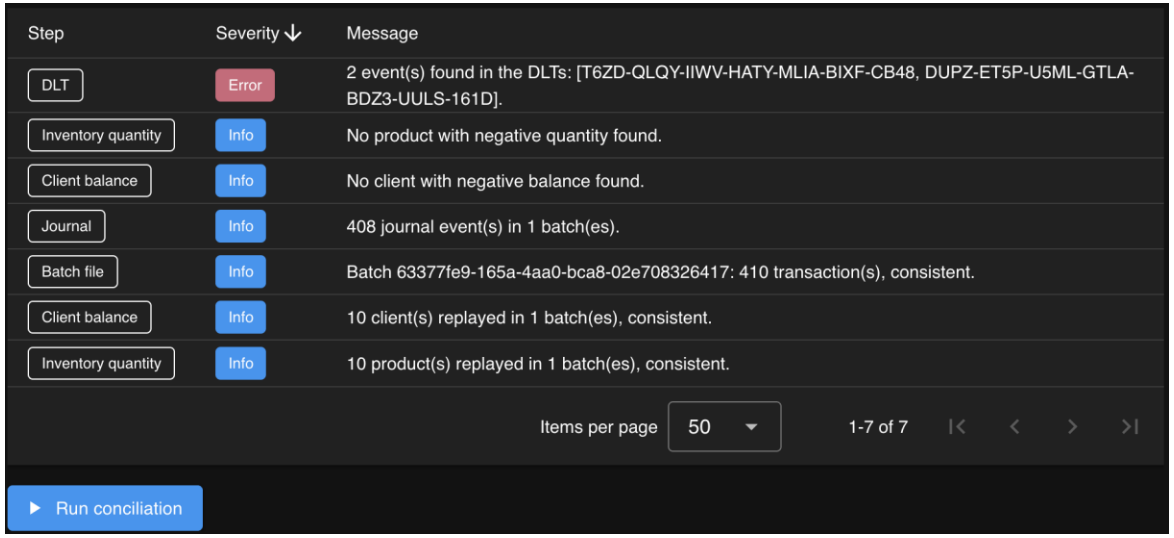
Réparer sans perdre la piste d'audit; rejouer sans réappliquer les effets en double.

Conciliation des données en EDA (2/2)

La conciliation consiste à rejouer/recalculer les mouvements effectués par la consommation des données.

Conciliation





Détecter, localiser, classifier, corriger et auditer (consigner).

Ce ne sont pas trois marches d'une échelle de maturité

La documentation compare des styles d'intégration complémentaires qui placent la complexité à des endroits différents.

Dimension	ETL (lots)	SOA (synchrone)	EDA / Kafka
Interaction	Tâches planifiées.	Requête-réponse.	Publication et consommation.
Valeur recherchée	Mouvements massifs et préparation.	Résultat immédiat pour le demandeur.	Consommateurs indépendants, tampon et rejeu.
Complexité à maîtriser	Calendriers, lignage et qualité.	Disponibilité, appels et compensation.	Ordre, doublons, DLT et cohérence à terme.

La bonne question : quel mécanisme répond au besoin avec une complexité maîtrisable ?

Justifier l'EDA par sa valeur, pas par le choix de la plateforme

Comparer les options sur le même horizon et expliciter les hypothèses, les coûts et les bénéfices.

TCO : le coût complet

Plateforme et environnements.
Ingénierie et exploitation.
Exactitude, tests et réconciliation.
Gouvernance et sécurité.

ROI : des bénéfices mesurables

Réaction plus rapide.
Réutilisation des événements.
Évolution indépendante.
Rejeu, audit et restauration.

Architecture : besoins + options + garanties + TCO/ROI + responsables + critères de révision.

Le sandbox rend les limites visibles; il ne les efface pas

La facilité d'exécution locale et les garanties d'une architecture de production sont deux objectifs distincts.

Pour apprendre localement	Pour aller vers une cible de production
Moteurs et base embarqués.	Services externes, état partagé et durabilité explicites.
Pont Flink in-JVM et stockage temporaire.	Runtime cible, connecteurs et restauration validés.
Options et tableaux de bord pédagogiques.	Sécurité, contrats, supervision et procédures opérables.
Anomalies volontaires de concurrence.	Invariants protégés et scénarios de défaillance testés.

Chaque garantie doit être rattachée à un périmètre, une hypothèse et une preuve.

Quatre expériences pour relier les concepts aux observations

Démo

Préparer un état de départ comparable, ne changer qu'une variable et expliquer ce qui est observé.

01 Avec / sans clé / sans lock

L'ordre par produit change-t-il l'inventaire ?

02 Rejeu et idempotence

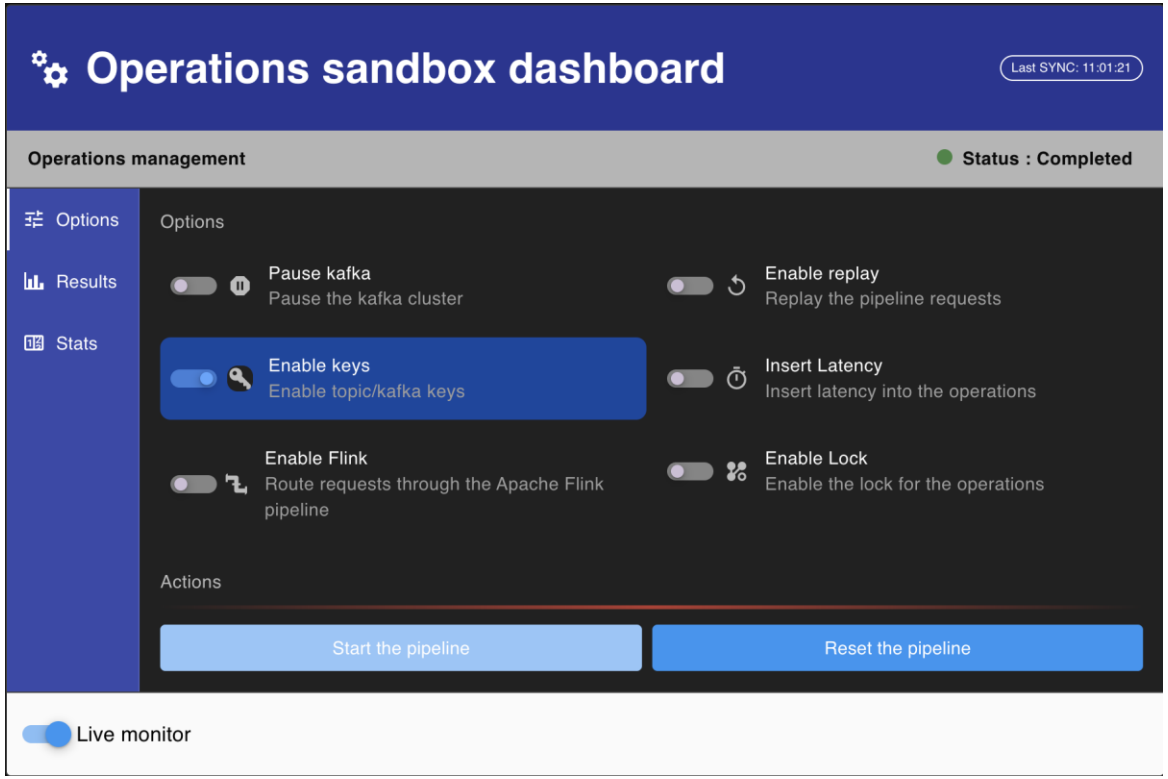
Quel effet ne doit pas être appliqué deux fois ?

03 Pause Kafka vs pause Flink

Que conserve chaque mécanisme de reprise ?

04 Vérification métier

Les requêtes expliquent-elles l'inventaires final ?



Observer → formuler une hypothèse → modifier une variable → expliquer le résultat (Effectuer une corroboration manuelle).

Une EDA résiliente se construit de bout en bout

La valeur du projet est de rendre visibles les responsabilités que le transport seul ne prend pas en charge.

Livrer un message, appliquer un effet et prouver le résultat sont trois responsabilités distinctes.

01 **Ordonner par entité**

02 **Délimiter l'atomicité**

03 **Tolérer les doublons**

04 **Préparer la reprise**

05 **Vérifier les données**

06 **Justifier la complexité**

Discussion : quelle garantie affirmez-vous, et comment la démontreriez-vous ?

Retrouver les chapitres du parcours dans le projet

Les chemins sont relatifs à doc/fr/. Chaque diapositive possède aussi ses sources détaillées dans les notes.

Présentation du projet

LISEZ_MOI.md

Architecture et topics

architecture_et_conception_des_topics.md

Données et journal

base_de_donnees.md

Comparaison des rapports

rapports.md

Livraison et fiabilité

semantiques_livraison_et_fiabilite.md

Acquittement et idempotence

reconnaissance_et_idempotence_consommateur.md

Persistance et transactions

patrons_persistance_et_transaction.md

Kafka, Flink et Hazelcast embarqués

guides_sandbox.md

Références de ce support : documentation du projet.

Poursuivre de la résilience technique à la confiance

La progression relie les mécanismes d'exécution, l'exploitation, la gouvernance et le choix d'architecture.

Chemin Flink du sandbox

fr/guides_flink.md

Cycle de vie et opérations

fr/cycle_vie_et_operations.md

Élasticité EKS

fr/elasticite_horizontale_eks.md

Corroboration

fr/verification_donnees.md

Choix ETL / SOA / EDA et TCO/ROI

fr/conclusion_etl_soa_eda_tco_roi.md

Intégration Flink / Spring Boot

fr/flink_springboot.md

Distribution et arrêt

fr/distribution_scale_et_arret.md

Gouvernance et observabilité

fr/gouvernance_et_observabilite.md

Data Mesh / Data Fabric

fr/datamesh_datafabric.md

Patrons de conception Inbox et Outbox

fr/Inbox_outbox_patterns.md

Références de ce support : documentation du projet.

Lire JRN_EVENT sans confondre ses responsabilités

Le journal distingue l'identité d'un traitement, sa corrélation, sa nature et son issue.

Champ	Question à laquelle il répond
JRN_EVENT__ID	Clé primaire. Est-ce la clé substitut ?
CONSUMER_ID + EVENT_ID	Quel consommateur a traité quel événement ?
BATCH_ID	À quelle publication appartient-il ?
EVENT_TYPE	Première livraison ou rejeu délibéré ?
RESULT	PENDING, BACK_ORDER, EXECUTED ou ERROR ?
PAYLOAD	Quel contenu et quelle chronologie de traitement ?

Les dépendances locales à remplacer en production

Le projet sandbox est un outil pédagogique.

Composant	Sandbox fourni	Cible distribuée
Base de données	HSQLDB fichier local	PostgreSQL / RDS partagé
Kafka	Broker embarqué, RF=1	Cluster externe, réplication ≥ 3
Hazelcast	TCP/IP localhost	Découverte Kubernetes + cohérence choisie
Flink	MiniCluster + pont JVM	Cluster externe + sink durable
Pilotage	Événements Spring locaux	Commandes distribuées et runId

Les limitations du sandbox

Le projet sandbox est un outil pédagogique. Certains choix pour faciliter l'apprentissage et/ou l'implémentation.

Limitation	Description
Pause	La fonctionnalité de la pause assume un seul processus. L'implémentation est à raffiner pour une PRD.
Replay	Le replay est très simpliste et vise une démonstration, pas une réelle implémentation. À raffiner pour une PRD.
Modèle SGBDR	Le modèle est volontairement simpliste. Client : Client, compte, solde, trx. Inventaire : Catégorie, modèle, etc.
Sandbox	Seul le Sandbox Kafka est qualité PRD. Flink est une base et Hazelcast nécessite du refactoring : Sandbox Datagrid.
Flink / In-JVM	Implémentation exploratoire in-JVM afin de ne pas avoir à refaire l'implémentation en SpringBoot.
Jeton / EVT	Afin de faciliter l'implémentation du « replay ». Le jeton ne contient pas le champ « version ».
Sécurité	La sécurité est simpliste, les valeurs sont codées en dur. Le jeton JWT est conservé dans le navigateur.